WoSM Academy

CS102-F26 · DIGITAL DESIGN AND COMPUTER ARCHITECTURE I

Logic and Comparison

Lecture 17: bitwise operations and comparators

Dr. Ali Khudiyev

Fall 2026

CC BY-SA 4.0

Today

01 Bitwise logic

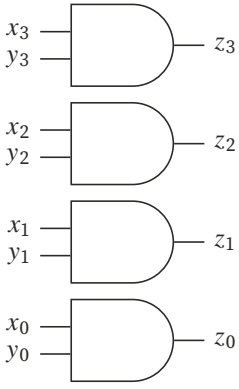
02 Comparing bits

03 Summary

01

Bitwise logic

One gate per bit



4-bit AND: $z_i = x_i \, y_i$

- Logic gates already do logic, one bit at a time (Unit 6). To AND two 8-bit numbers $x_{7:0}$ and $y_{7:0}$, give each pair of bits its own gate.
- The output is the 8 gate outputs side by side: $z_7 \ldots z_0$.
- The same for OR and XOR; NOT needs only one number.
- No bit affects any other: these are *bitwise* operations.

Bitwise operations at work

x	1100 1010
y	1010 0110
x AND y	1000 0010
x OR y	1110 1110
x XOR y	0110 1100
NOT x	0011 0101

- Work column by column: every column is a one-bit truth table.
- In the DX8: `and`, `or`, `xor` work on regA and regB, `not` on regA alone; the result goes to regOUT `(Unit 15)`.

What they are good for: masks

- A *mask* is a number chosen to act on some bits and leave the others alone.
- AND with a mask clears the bits where the mask is 0.
- OR with a mask sets the bits where the mask is 1.
- XOR with a mask flips the bits where the mask is 1.

x	`1011 0110`
AND `0000 1111`	`0000 0110`
OR `0000 1001`	`1011 1111`
XOR `0000 0001`	`1011 0111`
XOR `1111 1111`	`0100 1001`

XOR with all ones is NOT

> **Remark**
>
> To test one bit, AND with a mask that has only that bit set: the result is 0 exactly when the bit is 0. And x XOR $x = 0$ for every x.

02

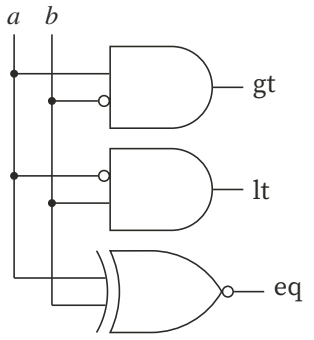
Comparing bits

Comparing two bits

a	b	gt	lt	eq
0	0	0	0	1
0	1	0	1	0
1	0	1	0	0
1	1	0	0	1

1-bit comparator (Table 6.5)

$$\text{gt} = a\,b', \quad \text{lt} = a'\,b, \quad \text{eq} = (a \oplus b)'$$



A bubble on an input inverts it

Comparing 4-bit numbers

- Compare as you would by hand: from the left. The first position where the numbers differ decides; if there is none, they are equal.
- One 1-bit comparator per position gives gt_i, lt_i, eq_i. Then:

$$\mathrm{eq} = \mathrm{eq}_3\,\mathrm{eq}_2\,\mathrm{eq}_1\,\mathrm{eq}_0$$
$$\mathrm{gt} = \mathrm{gt}_3 + \mathrm{eq}_3\,\mathrm{gt}_2 + \mathrm{eq}_3\,\mathrm{eq}_2\,\mathrm{gt}_1 + \mathrm{eq}_3\,\mathrm{eq}_2\,\mathrm{eq}_1\,\mathrm{gt}_0$$
$$\mathrm{lt} = \mathrm{lt}_3 + \mathrm{eq}_3\,\mathrm{lt}_2 + \mathrm{eq}_3\,\mathrm{eq}_2\,\mathrm{lt}_1 + \mathrm{eq}_3\,\mathrm{eq}_2\,\mathrm{eq}_1\,\mathrm{lt}_0$$

- Each term reads: “equal in every position to the left, and greater (or less) here”. This is the extra logic of the book’s 4-bit comparator: seven AND gates and two OR gates.

The comparator at work

position	3	2	1	0
$a = 1011$	1	0	1	1
$b = 1001$	1	0	0	1
eq_i	1	1	0	1
gt_i	0	0	1	0

- Positions 3 and 2 are equal; position 1 has $a_1 = 1 > b_1 = 0$.
- So the term $\text{eq}_3\,\text{eq}_2\,\text{gt}_1$ is 1, and gt = 1. Both lt and eq are 0.
- Position 0 does not matter any more.

8 bits

The same pattern with 8 positions: eq is the AND of 8 eq_i, and gt and lt have 8 terms each. The DX8 has an 8-bit comparator.

Signed numbers need care

- The comparator reads its inputs as unsigned numbers.
- In two's complement (`Unit 16`), 1111 1111 is −1 and 0000 0001 is 1, but the comparator finds 1111 1111 greater.
- Likewise 1000 0000 (−128) comes out greater than 0111 1111 (127).

> **One fix**
>
> Invert the leftmost bit of both numbers, then compare as unsigned: 0000 0000 < 1111 1111, so −128 < 127 as it should be. Inverting that bit adds 128 to every value, which keeps their order.

Comparison in the DX8

- `cmp` compares regA with regB `(Unit 15)`.
- The comparator's three outputs are the *flags* gt, lt and eq; the DX8 keeps them in regFLAG.
- `jmpgt`, `jmplt` and `jmpeq` jump only if their flag is set: this is how a DX8 program decides what to do next.

> **Remark**
>
> Exactly one of gt, lt and eq is 1 for any two inputs. Check it in the truth table of the 1-bit comparator, then in the equations of the 4-bit one.

Try it

1. $x = 0101\,1100$, $y = 0011\,1010$. Find x AND y, x OR y and x XOR y.
   $0001\,1000$, $0111\,1110$ and $0110\,0110$.
2. Which mask and which operation clear the left 4 bits of x and keep the right 4?
   AND with $0000\,1111$.
3. How do you flip only bit 0 of x?
   XOR with $0000\,0001$.
4. $a = 0110$, $b = 0101$. Which output of the comparator is 1, and which position decides?
   gt, decided at position 1 ($a_1 = 1$, $b_1 = 0$).
5. How many terms does gt have in an 8-bit comparator, and how many eq_i are in its longest term?
   8 terms; the longest has 7: $\mathrm{eq}_7 \cdots \mathrm{eq}_1\, \mathrm{gt}_0$.

03

Summary

Summary

- Bitwise AND, OR, XOR and NOT use one gate per bit; no bit affects another.
- Masks: AND clears bits, OR sets them, XOR flips them.
- A 1-bit comparator: gt = ab', lt = $a'b$, eq = $(a \oplus b)'$.
- An n-bit comparator: the leftmost position where the numbers differ decides. eq is the AND of all eq_i.
- The comparator treats numbers as unsigned. Its outputs are the DX8's flags for the conditional jumps.

Before the next lecture

- Read Section 6.3 of the book.
- In Logisim-evolution, build the 1-bit comparator and the 4-bit comparator, and test it on the pairs of this lecture. Then extend it to 8 bits.
- Build 8-bit AND, OR, XOR and NOT units.

> **Next lecture** (Unit 18)
>
> The arithmetic logic unit: shifts and rotations, and selecting the operation.